Tashkent University of Information Technologies

Course: Introduction to Software Engineering (ISWE)

PRACTICAL ASSIGNMENT 1

Software fundamentals, process models, Agile/Scrum and requirements engineering

Project: SmartBank

Digital banking system: registration by passport ID, card linking and OTP-confirmed card-to-card transfers

Student: Diyorxo'ja Saidaliyev

Group: 319-24

Team: 1 person

Tashkent, 2026

List of working links

Most project artefacts are stored in one public GitHub repository, **BankingSystem;** only the Trello board and the kick-off page are on other sites. Before each link the task and the content are named. All links are public (view access).

Table 1 – Working links to project artefacts

№	Task	What it is and where it leads	Link
1	Task 3.1	Life cycle diagram (PNG, with editable draw.io source) – GitHub repository, folder docs/diagrams	https://github.com/Diyor-S/BankingSystem/blob/main/docs/diagrams/life-cycle.png
2	Task 4.3	Trello board “SmartBank SB-2026” (5 columns, 11 User Stories, 8 task cards) – Trello	https://github.com/Diyor-S/BankingSystem/tree/develop/docs/trello
3	Task 4.3–4.5	Text copy of the Product Backlog, task cards and Sprint 1 plan (the same content as the Trello board) – GitHub	https://github.com/Diyor-S/BankingSystem/blob/main/docs/backlog-and-tasks.md
4	Task 4.4	Draft wireframes of the 3 transfer screens (task T-02) – GitHub	https://github.com/Diyor-S/BankingSystem/blob/main/docs/diagrams/wireframes.png
5	Task 5.2	Use Case diagram (PNG, with PlantUML and draw.io sources) – GitHub	https://github.com/Diyor-S/BankingSystem/blob/main/docs/diagrams/use-case.png
6	Task 5.3	Technical Specification (Markdown copy of Section 5.3) – GitHub	https://github.com/Diyor-S/BankingSystem/blob/main/docs/technical-specification.md
7	Task 5.3	Data model: ER diagram, SQL schema and mock-registry seed data – GitHub	https://github.com/Diyor-S/BankingSystem/tree/main/docs/database
8	Task 5.3	REST API description (OpenAPI) of the transfer flow and mock services – GitHub	https://github.com/Diyor-S/BankingSystem/blob/main/docs/api/openapi.yaml
9	Task 6	Page “Kick-off: SmartBank” – Google Docs	https://github.com/Diyor-S/BankingSystem/blob/develop/docs/kickoff.md
10	General	GitHub repository main page (README, all documents, later the code)	https://github.com/Diyor-S/BankingSystem

Introduction: how we resolve the kick-off situation

At the first meeting four stakeholders made conflicting demands: the client wants "everything competitors have" in 3 weeks; the developer has no specification; the designer has drawn 8 screens without scenarios; the marketer has already launched ads. As the project lead we treat all four statements as symptoms of one problem: **no shared goal, scope and plan**. In a banking project this problem is more dangerous than usual, because mistakes in money handling are expensive. The whole report fixes the problem step by step. Table 2 shows the link between each complaint, the root cause and the document in this report that resolves it.

Table 2 – Kick-off situation: complaint, root cause and our response

Who	Complaint	Root cause	Our response (where in the report)
Client	"Ready in 3 weeks, everything our competitors have."	Unlimited scope, no priorities, unrealistic deadline.	MVP scope with 8 features and an Out-of-scope list (Tasks 1.1, 6); MoSCoW priorities and Product Backlog (Task 4.3); bill payments, loans and passport scanning are postponed; only Sprint 1 (2 weeks) promises a working card-to-card transfer.
Developer	"No specification; where do we start?"	Requirements not written down.	11 User Stories, 13 FR and 7 NFR with traceability (Task 5.1), Technical Specification (Task 5.3), one key feature for Sprint 1 (Task 4.1).
Designer	"I drew 8 screens, which one do you like?"	Screens drawn without user scenarios.	Use Cases and user flow first (Task 5.2, T-01); wireframes only for the 3 screens of the Sprint 1 transfer flow (T-02). Other screens wait for their User Stories.
Marketer	"Ads are launched, I need something to show now."	Marketing ahead of the product.	Pause paid ads; use a landing page with a waiting list and the Sprint 1 demo (Task 6, communication plan). Advertising may only describe features that are Done: a bank must not promise services it does not provide.

Task 0. Project topic

Topic: SmartBank – a digital banking system for a commercial bank in Uzbekistan (hypothetical) that follows the typical modern flow. A person registers **only with a passport ID**: the system finds the passport in the state citizen registry and takes the full name and the phone number attached to it, and confirms the phone with an OTP (one-time SMS code). The customer then **links a bank card** – allowed only if the card is attached to the same phone number as the passport – and makes **card-to-**

card transfers confirmed by an OTP. The customer gets an SMS for every transaction and sees the history. An **admin** monitors users, cards and transactions and can freeze an account.

Mock external systems. We have no access to government or card-processing APIs, so three external systems are imitated by our own mock services with seeded test data: the **Passport Registry** (passport ID → full name, birth date, phone), the **Card Registry** (card number → attached phone, holder, balance, status) and the **SMS gateway** (stores the message and shows the OTP in a developer inbox). Real systems also read the passport with a camera; scanning is out of scope, the passport ID is typed in. The mocks are replaceable adapters, so real services can be connected later without changing the business logic.

Real-world problem. Opening an account and doing simple operations in many banks still means a branch visit, paper documents and waiting. Customers expect instant onboarding with a passport ID and a phone, and safe transfers confirmed by an OTP, as in the leading payment apps in Uzbekistan (public sources are in the References).

Table 3 – Check of the topic against the requirements of Task 0

Requirement	How SmartBank meets it
Specific problem, clearly defined users	Slow, branch-dependent onboarding and transfers. Users: customer and admin; external systems: passport registry, card registry and SMS gateway (mocks).
At least 2 user types	2 user types: customer and admin.
At least 8 features	8 features (Table 9, F1–F8) and 11 User Stories (Table 12).
Several interconnected modules	Registration, authentication (OTP), card linking, transfers, history, notifications, admin monitoring, and three mock services. A transfer depends on login, linked cards and OTP, and triggers notifications.
Not a simple CRUD application	Cross-checking passport, phone and card data; OTP lifecycle (expiry, attempt limits); atomic card-to-card transfer with balance checks, row locks and idempotency; audit log; account freezing.
Can be continued in later assignments	Enough material for UML and architecture, a Figma prototype, test plans and a final presentation.
Not repeated in the group	[To be confirmed: no other student of group 319-24 has chosen a banking topic.]

Task 1. Project charter and team roles

1.1. Project charter

Table 4 – Project charter of SmartBank

Item	Content
Project name and code	SmartBank, SB-2026
Client (hypothetical)	SmartBank JSC – a commercial bank in Uzbekistan (hypothetical); pilot with 2,000 selected customers. The client is imitated in a role-play (the student and, where possible, a reviewer).
Problem to be solved	Opening an account and making simple transfers requires a branch visit and paper work (assumed: onboarding takes 1–2 days and about 45 % of transfers are still made at branches). Customers expect to register with a passport ID and a phone and to transfer money safely online. These figures are assumptions to be measured at the start of the pilot.
Project goal	Within 3 months after the pilot launch, 2,000 customers register online (registration in ≤ 2 minutes) and make at least 70 % of their transfers through SmartBank, with zero balance discrepancies and a transfer processed in ≤ 3 seconds.
Main objectives	1. Registration with a passport ID checked against the (mock) citizen registry and confirmed by OTP. 2. Login with phone number and OTP. 3. Linking a card only if passport, phone and card match. 4. Card-to-card transfers confirmed by OTP, with an atomic balance update. 5. Transaction history and an SMS notification for every transaction. 6. Admin monitoring of users, cards and transactions, with account freezing.
Target users	Customers (adults with a passport, a phone and a bank card) and admins (bank staff who monitor the system).
Project boundaries (scope)	**Included:** responsive web app / PWA for customers; web panel for admins; mock Passport Registry, Card Registry and SMS gateway with seeded test data; UZS currency. **Not included:** passport scanning with a camera, biometrics, connection to real government and card-processing systems, real money, loans, bill payments, native mobile apps, manual KYC review, fraud-analysis tools.
Planned technologies	Python 3.12 + FastAPI (backend and mock services); PostgreSQL (ACID transactions); Redis (OTP storage with expiry, rate limiting); React + TypeScript (web / PWA); Docker; GitHub Actions (CI); Figma, Trello, Notion, draw.io; pytest, Playwright, Locust, OWASP ZAP; hosting on a server located in Uzbekistan.
Project timeline and phases	14 weeks: Phase 1 initiation and requirements (weeks 1–2); Phase 2 Sprints 1–4, 2 weeks each (weeks 3–10); Phase 3 system, security and acceptance testing (weeks 11–12); Phase 4 pilot launch and handover (weeks 13–14); maintenance afterwards.
Constraints and assumptions	**Constraints:** individual project of one student, about 20 hours per week; no access to real government or card APIs, so all external data come from mock databases; the prototype has no banking licence and uses test data only; a real launch would require integration with the real registries and legal review (Central Bank rules, AML/KYC rules, Law No. ZRU-547 “On Personal Data”). **Assumptions:** the phone number in the passport registry is the person’s current phone; every card is attached to a phone number in the card registry; customers have a smartphone and internet.
Success criteria	2,000 registered pilot customers in 3 months; ≥ 70 % of their transfers made digitally; 0 balance discrepancies in daily reconciliation; transfer time ≤ 3 s (95th percentile); registration ≤ 2 minutes; availability ≥ 99.5 %; usability score SUS ≥ 75.

1.2. Team roles

This is an individual project: **Diyorxo'ja Saidaliyev** holds all roles. In a real company these roles belong to different people, so the table also shows what each role is responsible for, so that the work can be planned and defended role by role. The allocation is the same in Sections 4.2 and 5 and in the TS.

Table 5 – Roles in the SmartBank project

Role	Area of responsibility	Specific task in the project	Performed by
Client (CEO / sponsor)	Defines business needs, pays, accepts the result.	SmartBank JSC: approves scope, provides the rules for test data, signs the acceptance act.	Hypothetical client (role-play)
Product Manager	Product vision, market analysis, roadmap, KPIs.	Analyses competitors and monetisation, defines the roadmap and success metrics (Tasks 2, 6).	Diyorxo'ja Saidaliyev
Project Manager	Time, resources, risks, communication with the client.	Project charter, risk register, weekly client report, deadline control.	Diyorxo'ja Saidaliyev
Product Owner	Owns and prioritises the Product Backlog; accepts Sprint results.	Writes User Stories with acceptance criteria, sets MoSCoW priorities, accepts work at Sprint Review.	Diyorxo'ja Saidaliyev
Scrum Master	Facilitates Scrum events, removes blockers.	Runs daily stand-ups, Sprint Planning, Review and Retrospective; maintains the Trello board.	Diyorxo'ja Saidaliyev
Developer (backend)	Server side, database, APIs, mock services.	Transfer API with atomic balance update and idempotency; OTP logic; mock Passport, Card and SMS services.	Diyorxo'ja Saidaliyev
Developer (frontend)	Client side of the system.	React transfer page, registration, card-linking and admin screens, error handling.	Diyorxo'ja Saidaliyev
UI/UX Designer	User research, flows, wireframes, prototype.	User flow of the transfer, wireframes and a clickable prototype based on use cases (not before).	Diyorxo'ja Saidaliyev
Tester (QA)	Test design, manual and automated tests, security checks, bug reports.	15 manual test cases for transfers; automated 50-parallel-transfer test; OWASP ZAP scan; UAT preparation.	Diyorxo'ja Saidaliyev
Marketer / Business analyst	Communication with potential users and partners.	Landing page with waiting list; interviews with customers about onboarding and transfers; preparation of mock data rules.	Diyorxo'ja Saidaliyev

Task 2. Product classification and market analysis

2.1. Software type

SmartBank is a **transactional web application (financial information system)** that depends on external registries (imitated by mock services) and has a **soft real-time component** (OTP delivery and notifications). The justification:

- **Web / PWA, not system or embedded software:** customers and admins work in a browser; nothing is installed and no hardware is controlled.
- **Transaction-processing (OLTP) system:** the main value is correct handling of money: every transfer must be atomic, consistent, isolated and durable (ACID), and no balance may become negative.
- **Security-critical:** the system stores personal data (passport ID, phone, card number), so OTP authentication, access by role, masking, encryption and audit are core requirements.
- **Integration-based, soft real-time:** the flow depends on a passport registry, a card registry and an SMS gateway (mocks in this project); an OTP must reach the user in ≤ 10 seconds and a transfer must finish in ≤ 3 seconds (NFR-08), but a short delay does not cause damage, so it is soft, not hard, real-time.

2.2. Monetisation models

Table 6 – Comparison of monetisation models for SmartBank

Model	Essence	Pros	Cons	Company examples	Key metrics
Freemium	Basic account and transfers free; extra services paid.	Fast customer growth; low entry barrier.	Low conversion to paid; free users still cost money.	Revolut, Monzo	Conversion rate, MAU, ARPU
Transaction / interchange fees	Fee for card payments, bill payments and some transfers.	Income grows with usage; customers pay only when they use the service.	Fees are limited by competition and regulation; free P2P in payment apps.	Card issuers, Wise	Transaction volume, take rate
Net interest margin (loans and deposits)	Earn the difference between loan and deposit interest.	The largest revenue source of a bank.	Credit risk; needs capital, licence and scoring.	Traditional and digital banks	NIM, NPL ratio, loan book
Subscription (premium packages)	Monthly fee for premium features and service.	Predictable income.	Customers must see clear value; low adoption in the mass market.	Revolut Premium	MRR, churn

Model	Essence	Pros	Cons	Company examples	Key metrics
Partner offers / advertising	Partners pay for cashback offers or referrals.	Extra income without charging customers.	Privacy and trust concerns; unstable income.	Marketplace-based banks	Click-through rate, partner revenue
White-label licensing (BaaS)	Sell the platform to other banks.	Scalable; recurring licence fees.	Long sales cycle; heavy support; needs a mature product.	Temenos, Solaris	Number of client banks, ARR

Chosen model: free basic digital account (freemium) monetised by transaction and interchange fees; lending margin in a later phase. The pilot is funded from the bank's digital-transformation budget. Arguments:

- **Customers compare with free alternatives.** Payment apps already offer cheap or free P2P transfers, so a paid basic account would block adoption; the goal of 2,000 pilot customers depends on a free start (in the MVP transfers carry no fee; fee rules are a later configuration).
- **Revenue grows with usage.** Interchange on card payments and fees for payments and interbank transfers scale with the share of digital operations, which is exactly the goal (70 % of transfers digital), and branch cost per operation falls at the same time.
- **Lending is the biggest income but needs trust and risk control.** Online loans are outside this release; they come after onboarding, card linking and transfers have proved reliable.

2.3. Competitors and analogues

Table 7 – Analysis of existing solutions

Solution	Strengths	Weaknesses (hypotheses to validate by interviews)	How SmartBank differs
Click (SuperApp, Uzbekistan)	Very large user base (public data: over 20 million registered users); P2P transfers between Uzcard and Humo cards; utility, tax and service payments; integration with government services.	Payment-centric: accounts and cards belong to partner banks, so the main banking relationship stays elsewhere; a bank cannot control the whole customer journey.	SmartBank is the bank itself: registration from the passport registry, card linking with a passport–phone–card check, own transfer ledger and an admin monitoring panel; the first release is small but fully controlled.
Uzum Bank	Part of a large ecosystem (marketplace + fintech); free virtual and plastic cards; cashback; QR payments; transfers between Uzcard and Humo.	Value is tied to the Uzum ecosystem; a mid-size bank cannot copy the marketplace advantage.	Focus on trust and transparency: clear OTP confirmation of every transfer, instant notifications, a simple flow for the bank's own customers.

Solution	Strengths	Weaknesses (hypotheses to validate by interviews)	How SmartBank differs
TBC Bank UZ / Payme	Digital-first bank with consumer lending; Payme is among the leading P2P and payment apps.	Strong scale and brand; hard to compete on features and marketing budget.	Do not compete on breadth: deliver a smaller MVP (passport-ID registration, linked cards, OTP-confirmed transfers) that a bank can build and verify quickly.

Task 3. Life cycle and process model

3.1. Life cycle diagram

Figure 1 shows the life cycle of SmartBank. Each blue box names a stage and the main work done in this project; the green boxes show which stakeholders the team talks to at that stage and about what; the orange frame marks the **current stage – requirements and planning** (this assignment), with three key risks below it. The purple arrow shows that every Sprint and every change request goes back to the requirements stage. The editable source of the diagram is in the link list.

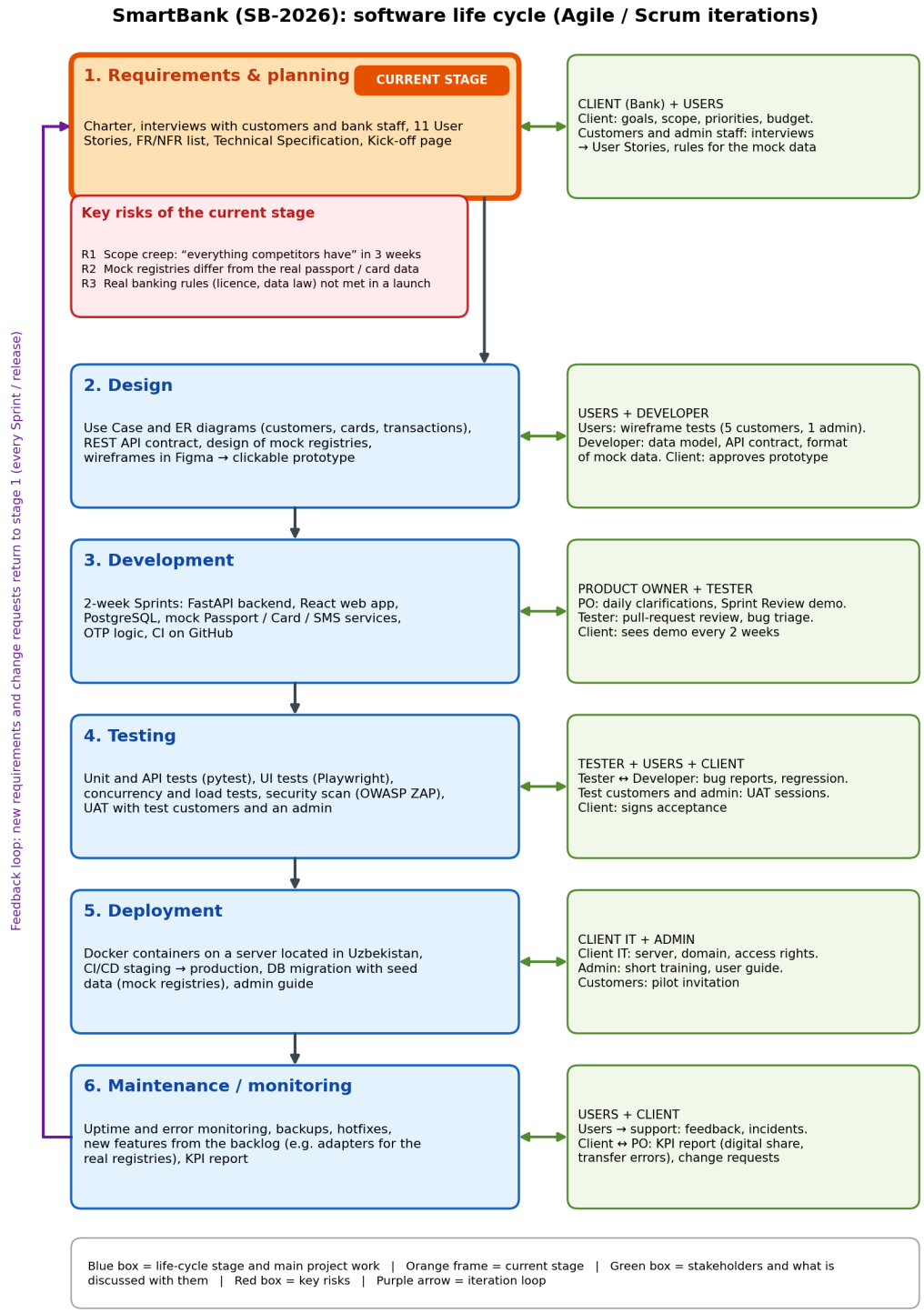


Figure 1 – Life cycle of SmartBank with the current stage, key risks and stakeholder communication

3.2. Choosing a process model

Table 8 compares five models from the point of view of SmartBank: a one-person student project, a short deadline (Sprint 1 in 2 weeks, pilot in 14 weeks), unclear and changing requirements, and a system where errors in money handling and security have serious consequences.

Table 8 – Comparison of process models for SmartBank

Criterion	Waterfall	V-model	Spiral	Iterative	Agile (Scrum)
Adaptability to changing requirements	Low – requirements are frozen at the start; a change restarts the chain.	Low – changes must be reflected in every test level.	Medium – new loops allow change, but each loop is heavy.	Medium – changes enter the next iteration.	High – the backlog is re-prioritised every Sprint.
Safety and reliability	Medium – thorough documents, but defects found late.	High – each phase has a matching test level; suited to critical systems.	High – risk analysis in every loop.	Medium – depends on the tests of each iteration.	Medium – needs a strong Definition of Done, CI and security checks to compensate for light documents.
Role of testing	Low – testing is a late phase.	High – test plans are written together with each design level.	Medium – tested at the end of each loop.	Medium – tested after each iteration.	High – testing in every Sprint, QA tasks on the board.
Risk management	Low – risks appear at the end.	Medium – risks found by verification early.	High – explicit risk analysis in each loop.	Medium – risks found after each iteration.	High – risks reviewed in every Planning and Retrospective; short feedback.
Communication with the client	Low – at the beginning and at the end.	Low – mostly at phase gates.	Medium – at each loop review.	Medium – after each iteration.	High – client takes part in Sprint Review every 2 weeks.
Fit with project size and timeline	Low – long, rigid plan for a small team and short deadline.	Low – too much documentation for one person.	Medium – risk workshops are costly for a small team.	Medium/High – fits, but has no fixed roles and rituals.	High – small team, 2-week Sprints, early working result.

Chosen model: Agile with the Scrum framework (2-week Sprints), enriched with Spiral-style risk review at every Sprint Planning and V-model-style test planning and a security review inside the Definition of Done. Justification:

- **Requirements are unclear and will change.** The client says “everything competitors have”, and the pilot will show which services customers really use. Scrum lets us re-prioritise the Product Backlog every 2 weeks; Waterfall and the V-model would fix requirements too early.
- **The client needs an early demonstration.** Investors and bank management want to see a working result soon. Scrum delivers a working money transfer at the end of Sprint 1, and the Sprint Review gives regular client feedback; a Waterfall result would appear only at the end.
- **The team is one person and the deadline is short.** With one person, heavy documentation and long risk workshops (Spiral, V-model) would take more time than the development. Scrum has a few roles and events that suit a small team, and Trello is enough for tracking.

- **Correctness and security can still be controlled.** The main weakness of Agile for a bank (lighter documents and quality control) is reduced by the Definition of Done (reviewed code, automated tests, security check), concurrency and parallel-transfer tests, a threat model, a Technical Specification updated every Sprint and the risk table of the kick-off page.

Task 4. Scrum: planning the first Sprint

4.1. Selecting a feature

Table 9 evaluates the eight features of the project (F1–F8) and two features that were considered and excluded (F9, F10). Priority is on a 1–5 scale (5 = most important), based on alignment with the project goal, dependency of other features and value for users.

Table 9 – Candidate features of SmartBank and their evaluation

Feature	Alignment with project goal	Complexity (Low / Med / High)	Priority (1–5)	Reason for selection / postponing
F1. Registration by passport ID (lookup in the mock Passport Registry, OTP to the registered phone)	High – instant onboarding is part of the goal.	Medium	5	Enabler of everything; Sprint 2.
F2. Login with phone number and OTP	High – secure access.	Low	5	Enabler; Sprint 2. Sprint 1 uses seeded test customers.
F3. Link a card (passport, phone and card must match; OTP confirmation)	High – no transfers without linked cards.	Medium	4	Sprint 2; Sprint 1 uses seeded linked cards.
F4. View linked cards and balances	High – customers must see their money.	Low	4	Minimal list is part of Sprint 1; full version in Sprint 3.
F5. Card-to-card transfer confirmed by OTP	Very high – the project goal is 70 % of transfers made digitally.	High	5	Core of the product; every other feature supports it. **Selected for Sprint 1.**
F6. Transaction history	Medium – transparency.	Low	3	Needs finished transfers; Sprint 3.
F7. SMS notification for each transaction	High – trust and early detection of problems.	Low	3	Quick to add after transfers exist; Sprint 3.
F8. Admin monitoring (users, cards, transactions; freeze / unfreeze account)	Medium – control for the bank.	Medium	3	Needs data from other features; Sprint 3–4.
F9. Bill payments (excluded)	Low for the MVP.	High	1	Needs a payment-network integration; “Won’t” in this release.
F10. Passport scanning with the camera (excluded)	Low – typing the passport ID is enough.	High	1	Out of scope: needs image recognition and device access.

Feature chosen for Sprint 1: F5 – card-to-card transfer. Arguments:

- **It is the core of the project goal.** The goal is measured by the share of transfers made digitally; notifications, history and admin monitoring all use transfer data.
- **It gives a working result that can be shown in 2 weeks** to the client, investors and the marketer’s audience: a test customer sees their cards, enters a recipient

card and an amount, and the balances change, which answers the client's demand for something visible.

- **It removes the main technical risk early.** Atomic debit and credit, protection against duplicate and parallel requests and balances that never become negative are the hardest consistency problems of the system; solving them first protects all later work.

4.2. Scrum roles and Agile values

Table 10 – Scrum roles (consistent with Table 5)

Scrum role	Person	Responsibility in SmartBank
Product Owner	Diyorxo'ja Saidaliyev	Owns the Product Backlog, writes User Stories and acceptance criteria, sets priorities with the client, accepts or rejects results at Sprint Review.
Scrum Master	Diyorxo'ja Saidaliyev	Facilitates Sprint Planning, daily stand-up, Review and Retrospective; removes blockers; keeps the Trello board up to date.
Development Team	Diyorxo'ja Saidaliyev (a team of one: backend, frontend, design, QA)	Turns the Sprint Backlog into a working increment that meets the Definition of Done.

Because this is an individual project, one person holds all Scrum roles (the same allocation as in Table 5). To keep the roles separate in practice, the work is organised by "hats": priorities are decided only at Sprint Planning and Review (Product Owner hat), daily work is done as Developer, and the Retrospective is written as Scrum Master. The Sprint Review demo is shown to the client's role-player to get independent feedback.

Table 11 – Agile Manifesto values in SmartBank

Agile value	How it shows in our project
Individuals and interactions over processes and tools	Instead of a formal approval process, the 8 drawn screens are checked in a 30-minute session with one real user (a classmate); the daily stand-up is a short written note, and Trello is only a support tool.
Working software over comprehensive documentation	Sprint 1 ends with a working card-to-card transfer. The TS and wireframes cover only what is needed for the next Sprints and are updated in each Sprint instead of being written "perfectly" at the beginning.
Customer collaboration over contract negotiation	The client's role-player and one test user join the Sprint Review every 2 weeks and re-prioritise the backlog with us, instead of fixing every feature in a contract.

Agile value	How it shows in our project
Responding to change over following a plan	If interviews show that customers care more about notifications than about history, US-07 moves up in the backlog at the next Sprint Planning. Changes are accepted between Sprints, not inside a Sprint.

4.3. Product Backlog and Trello board

The Trello board “SmartBank SB-2026” has five columns: Backlog → To Do → In Progress → Testing → Done. Figure 2 shows its state at Sprint 1 planning on Day 1: all user stories and the 8 task cards of Section 4.4 are in Backlog; the three Sprint 1 tasks and the key User Story US-01 are in To Do. Labels: orange – User Story, purple – Design, blue – Dev, green – QA. The public link is in the link list. **Before submission, replace Figure 2 with a real screenshot of the team’s Trello board** (the picture here shows the intended structure).

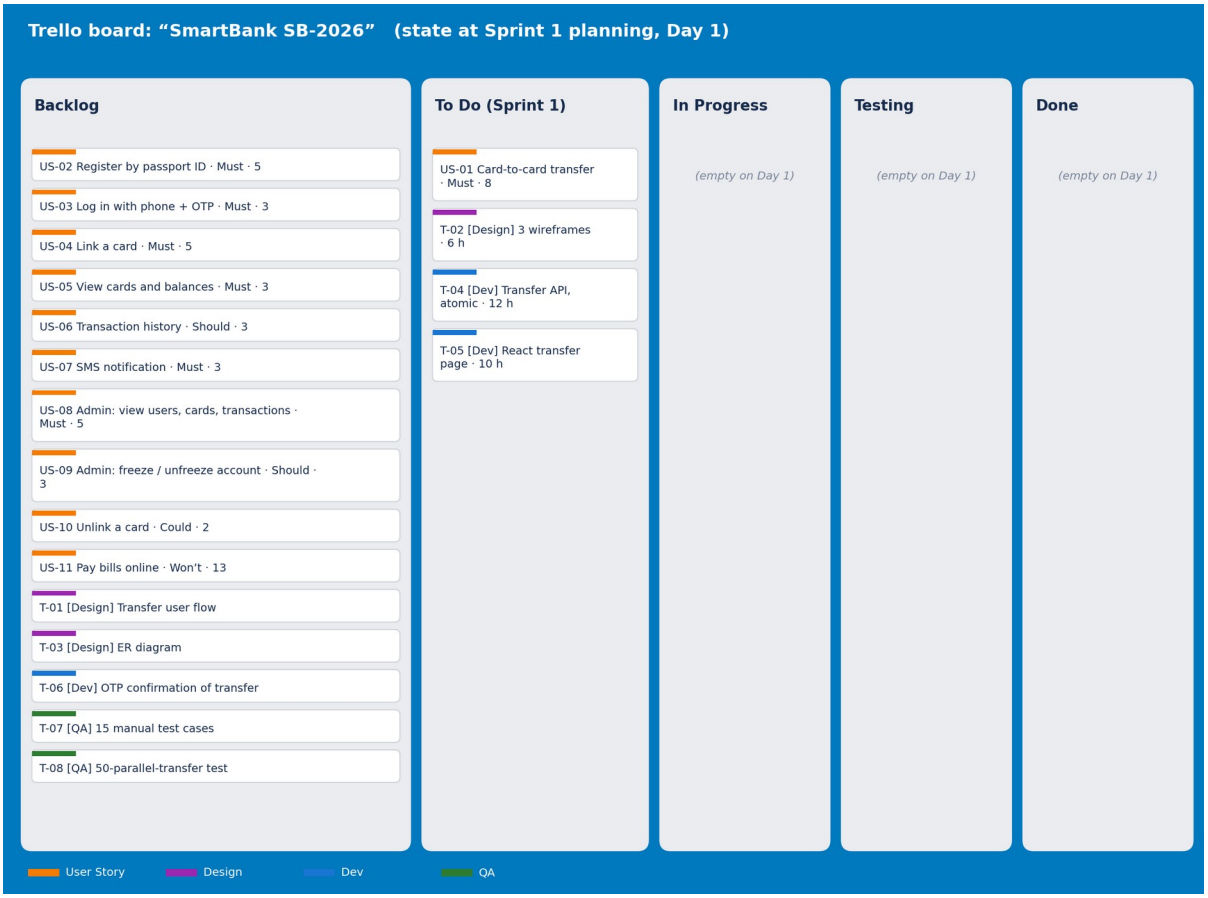


Figure 2 – Trello board “SmartBank SB-2026” at Sprint 1 planning

Table 12 lists the Product Backlog. Priorities follow MoSCoW, estimates use the Fibonacci Story Point scale (1, 2, 3, 5, 8, 13).

Table 12 – Product Backlog: User Stories with acceptance criteria

ID	User Story	Acceptance criteria	Priority	Story Point
US-01	As a customer, I want to transfer money from my linked card to another card by its number, because I do not want to visit a branch or carry cash.	1) The transfer is rejected with “insufficient funds” (HTTP 422) if balance < amount. 2) A recipient card that does not exist or is blocked is rejected (HTTP 404). 3) Debit and credit are written in one atomic transaction; there is never a partial transfer. 4) The transfer is executed only after the correct OTP is entered. 5) A repeated request with the same idempotency key does not create a second transfer. 6) A receipt with a reference number is shown in ≤ 3 s.	Must	8
US-02	As a new customer, I want to register by entering only my passport ID, because I do not want to fill in forms or visit a branch.	The passport ID is found in the Passport Registry (mock); the full name and phone are taken from it; an OTP is sent to that phone; after the correct code the account is created; one account per passport; unknown passport ID is rejected.	Must	5
US-03	As a customer, I want to log in with my phone number and an OTP, because I do not want to remember a password.	6-digit code valid 3 minutes; maximum 3 attempts, then a 15-minute pause; session ends after 5 minutes of inactivity.	Must	3
US-04	As a customer, I want to link my bank card to my profile, because I need a source of money for transfers.	The card exists and is active in the Card Registry (mock); the phone attached to the card is equal to my phone from the passport; the card is not linked to another customer; an OTP confirms the linking; otherwise the reason is shown.	Must	5
US-05	As a customer, I want to see my linked cards and balances, because I need to know how much money I have.	List of cards with masked numbers (last 4 digits) and balances in UZS opens in ≤ 2 s; balance equals the card registry value.	Must	3
US-06	As a customer, I want to see my transaction history, because I want to check my past transfers.	List of the last 12 months, newest first, with date, amount, counterpart (masked) and status; filter by date; opens in ≤ 2 s.	Should	3
US-07	As a customer, I want to receive an SMS for every transaction, because I want to notice unauthorised operations quickly.	SMS to the sender and the recipient ≤ 10 s after the transaction; text has amount and masked card; exactly one SMS per participant.	Must	3
US-08	As an admin, I want to view all users, their cards and transactions, because I must monitor the system.	Lists with search by passport ID, phone or card; each user page shows linked cards and transactions; every view of personal data is written to the audit log; customers cannot open admin pages.	Must	5
US-09	As an admin, I want to freeze and unfreeze a customer’s account, because suspicious accounts must be stopped quickly.	After freezing, all debit operations of the customer are declined and the customer sees the reason; unfreezing restores them; action and admin name are written to the audit log.	Should	3
US-10	As a customer, I want to unlink a card, because I no longer use it.	Unlinking needs an OTP; history stays; the card can be linked again later.	Could	2
US-11	As a customer, I want to pay bills online from my card, because I do not want to queue.	Not planned for this release (needs a payment-network integration).	Won’t	13

4.4. Breaking the feature into tasks (decomposition)

Feature F5 (US-01, card-to-card transfer) is divided into 8 tasks: 3 design, 3 development and 2 testing tasks. Every task is a separate Trello card with an owner, an estimate and a measurable result.

Table 13 – Decomposition of the transfer feature into tasks

ID	Task (outcome-oriented)	Owner	Estimate	Sprint
T-01	[Design] Draw the end-to-end transfer user flow in draw.io (choose card → enter recipient card and amount → confirm → enter OTP → receipt) with at least 8 steps and 3 error branches (insufficient funds, wrong OTP, recipient not found).	Diyorxo'ja Saidaliyev	3 h	2
T-02	[Design] Create low-fidelity wireframes (Figma or draw.io) for the 3 screens of the transfer flow – card list with balances, transfer form (recipient card, amount), OTP entry with receipt – and check them with 2 potential users.	Diyorxo'ja Saidaliyev	6 h	1
T-03	[Design] Draw the ER diagram with 6 tables (customers, cards, transactions, otp_codes, admins, audit_log), keys and the constraints CHECK(balance ≥ 0) and UNIQUE(idempotency_key).	Diyorxo'ja Saidaliyev	3 h	2
T-04	[Dev] Implement POST /api/transfers (debit and credit in one DB transaction with row locks and an idempotency key; returns 201, 404 for an unknown recipient card, 409 for a duplicate key or 422 for insufficient funds) and GET /api/cards with balances; includes the DB migration and seed data of 5 test customers with 8 cards in the mock Card Registry.	Diyorxo'ja Saidaliyev	12 h	1
T-05	[Dev] Build the React page “Transfer money” (card selector, recipient card field, amount, confirmation dialog, receipt, error messages for insufficient funds, unknown card and duplicate) connected to the endpoints of T-04; works from 360 px screen width.	Diyorxo'ja Saidaliyev	10 h	1
T-06	[Dev] Add OTP confirmation to the transfer: generate a 6-digit code, send it through the mock SMS gateway, valid 3 minutes, maximum 3 attempts; the transfer is executed only after the correct code.	Diyorxo'ja Saidaliyev	6 h	2
T-07	[QA] Write and run 15 manual test cases (TC-01…TC-15) for transfers: normal flow, insufficient funds, zero or negative amount, same card, unknown recipient, blocked card, double click, wrong OTP, 360 px layout.	Diyorxo'ja Saidaliyev	5 h	2
T-08	[QA] Write an automated pytest test that sends 50 parallel transfers of 100,000 UZS from a card holding 1,000,000 UZS and checks that exactly 10 succeed, 40 are rejected, the final balance is 0, it never goes negative and the total money of all cards is unchanged.	Diyorxo'ja Saidaliyev	3 h	2

Figure 3 shows the draft wireframes of the 3 screens of task T-02 (card list with balances, transfer form, OTP entry); the OTP screen is added to the build in Sprint 2.

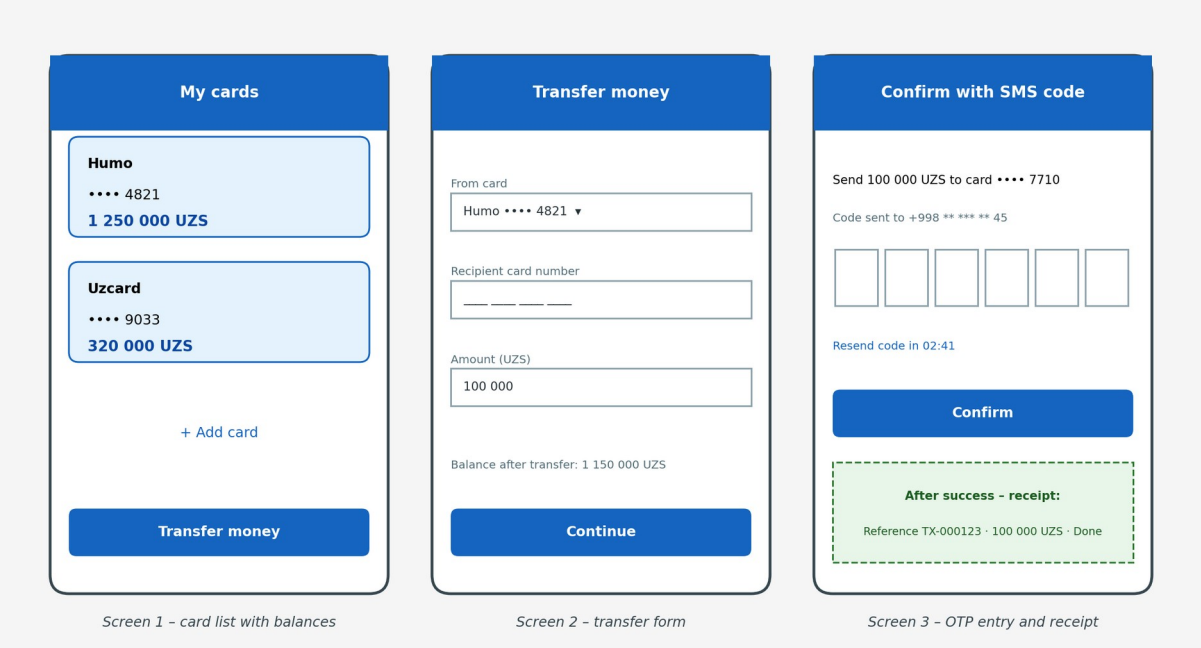


Figure 3 – Draft wireframes of the transfer flow (task T-02)

4.5. Sprint plan

Sprint 1 (2 weeks, weeks 3–4). Capacity: 1 person × 20 h per week × 2 weeks = 40 h; the selected tasks need 28 h, and the remaining 12 h (30 %) are a buffer for self-review, testing, the demo and unexpected problems.

Sprint goal: By the end of Sprint 1 a seeded test customer can open the web page, see their cards with balances and transfer money from a card to another card by card number; the transfer is stored atomically, the balance never becomes negative and a repeated request does not create a second transfer.

Table 14 – Tasks moved to the To Do column for Sprint 1

ID	Task	Owner	Estimate	Why it is in Sprint 1
T-02	[Design] Wireframes of 3 transfer screens	Diyorxo'ja Saidaliyev	6 h	The frontend needs agreed screens; this fixes the “8 screens without scenarios” problem and stops design work on unrelated screens.
T-04	[Dev] Transfer API with atomic balance update	Diyorxo'ja Saidaliyev	12 h	Critical path and the biggest technical risk; nothing can be shown without it.
T-05	[Dev] React transfer page	Diyorxo'ja Saidaliyev	10 h	Gives users and the client something visible and usable at the end of the Sprint.

Why these three tasks. Together they form the shortest path to a usable result: T-02 defines what the user sees, T-04 provides the data and the money rules, T-05 connects them in a page. T-01 and T-03 document what the Sprint already implements; T-06 (OTP confirmation) and the two QA tasks improve and verify the

result but do not block the demonstration; they start in Sprint 2, when US-01 is completed with the OTP step. For Sprint 1 the users and cards are seeded test data with a temporary login (until US-02 and US-03 in Sprint 2) and all money is test money. Quality is controlled by the Definition of Done below.

Definition of Done – a task is “done” when:

- The code is merged into the main branch by a pull request after a self-review checklist (the diff is read again the next day) and passing CI checks; a classmate may review it if available.
- Unit and API tests of the task pass in CI (GitHub Actions) and cover at least 70 % of the new code; for money operations the concurrency test also passes.
- The acceptance criteria of the linked User Story are verified by the Product Owner on the staging server during a live demonstration.
- A security check is done: no secrets in the code, input is validated, access is by role, personal data are masked, and no High or Critical finding remains from the automated scan.
- There are no open Critical or High bugs related to the task.
- The API is documented in OpenAPI/Swagger and the README describes how to run the result (for design tasks: the Figma link is added to the card and the screens were checked with 2 potential users).
- The Trello card is moved to Done only after the Product Owner’s acceptance.

Task 5. Requirements and technical specification

5.1. Functional and non-functional requirements

Every functional requirement is linked to one User Story of Table 12 (traceability). US-11 is “Won’t” and therefore has no requirement.

Table 15 – Functional requirements

ID	Name	Description	Priority	Source
FR-01	Passport lookup	The system must look up the entered passport ID in the Passport Registry (mock database), take the full name and the attached phone number from it, and reject an unknown passport ID or a passport that already has an account.	High	US-02
FR-02	Registration by OTP	The system must send a 6-digit OTP to the phone from the registry and create the customer account only after the correct code is entered.	High	US-02
FR-03	Login by OTP	The system must log in a customer by phone number and an OTP valid for 3 minutes, allow at most 3 attempts (then a 15-minute pause) and end the session after 5 minutes of inactivity.	High	US-03
FR-04	Card linking check	The system must link a card only if it exists and is active in the Card Registry, is not linked to another customer and its attached phone number equals the customer’s phone number; otherwise it must show the reason.	High	US-04
FR-05	Cards and balances	The system must show the customer’s linked cards with masked numbers (last 4 digits) and current balances.	High	US-05
FR-06	Card-to-card transfer	The system must transfer an amount from the customer’s linked card to a recipient card: check that both cards are active, the amount is positive and the balance is sufficient, and write debit and credit in one atomic transaction; it must return a receipt with a reference number.	High	US-01
FR-07	OTP confirmation of transfer	The system must send an OTP to the customer’s phone for every transfer and execute the transfer only after the correct code is entered (maximum 3 attempts).	High	US-01
FR-08	Duplicate protection	The system must accept an idempotency key with every transfer request and return the original result if the same key is sent again, without a second debit.	High	US-01
FR-09	Transaction history	The system must show the customer’s transactions of the last 12 months, newest first, with a filter by date.	Medium	US-06
FR-10	Notifications	The system must send an SMS to the sender and the recipient within 10 seconds after each transaction, with the amount and the masked card number.	High	US-07
FR-11	Admin monitoring	The system must give an admin lists of users, their cards and transactions with search by passport ID, phone or card, record every view of personal data in the audit log and deny admin pages to customers.	High	US-08
FR-12	Freeze / unfreeze account	The system must let an admin freeze and unfreeze a customer’s account; a frozen account cannot make debit operations; every action is audited.	Medium	US-09
FR-13	Unlink card	The system must let a customer unlink a card after an OTP confirmation and keep the history.	Low	US-10

Table 16 – Non-functional requirements

ID	Category	Requirement	Metric
NFR-01	Performance	Pages with cards, balances and history must respond quickly under normal load.	95th percentile ≤ 2 s with 100 concurrent users
NFR-02	Reliability	The system must be available during the pilot and protected against data loss.	Availability ≥ 99.5 % per month; daily backup; restore ≤ 4 h
NFR-03	Security	All traffic is encrypted; OTPs are random, short-lived and stored only as hashes; personal data are masked in the interface and encrypted at rest; access depends on the role; every admin action is audited.	HTTPS TLS ≥ 1.2; OTP valid ≤ 3 min, ≤ 3 attempts; passport ID and card number AES-256 at rest; 100 % of admin actions in the audit log; 0 High/Critical findings in an OWASP ZAP scan
NFR-04	Usability	A first-time customer must be able to register and transfer money without training.	Registration ≤ 2 min; transfer ≤ 5 steps and ≤ 60 s; SUS ≥ 75 in a test with 10 users
NFR-05	Scalability	The system must serve the pilot and allow growth without redesign.	10,000 customers, 20 transfers per second at peak; load test 200 concurrent users with p95 ≤ 3 s
NFR-06	Data integrity	Money must never be lost, created or duplicated, and balances must not become negative.	0 negative balances and 0 duplicated transfers in 1,000 parallel requests; total money of all cards unchanged by transfers
NFR-07	Compatibility	The web app must work on common devices and browsers.	Last 2 versions of Chrome, Safari, Firefox; Android 9+, iOS 15+; layout from 360 px width
NFR-08	Performance (real time)	Transfers and OTP / SMS delivery must be fast.	Transfer processed ≤ 3 s; OTP and notification sent ≤ 10 s after the event

5.2. Use Case diagram

Figure 4 shows the Use Case diagram. **Actors:** Customer and Admin (human users) and three external systems that are imitated by mock services: Passport Registry, Card Registry and SMS gateway. There are 12 use cases; relationships «include» (Register includes Verify passport; Link card includes Check passport–phone–card match and Confirm operation by OTP; Transfer money includes Confirm operation by OTP and Send notification) and «extend» (Freeze / unfreeze account extends View users, cards and transactions). Login and registration use an OTP as their own main flow, so the shared “Confirm operation by OTP” is included only by the operations that confirm an action of an already logged-in customer.

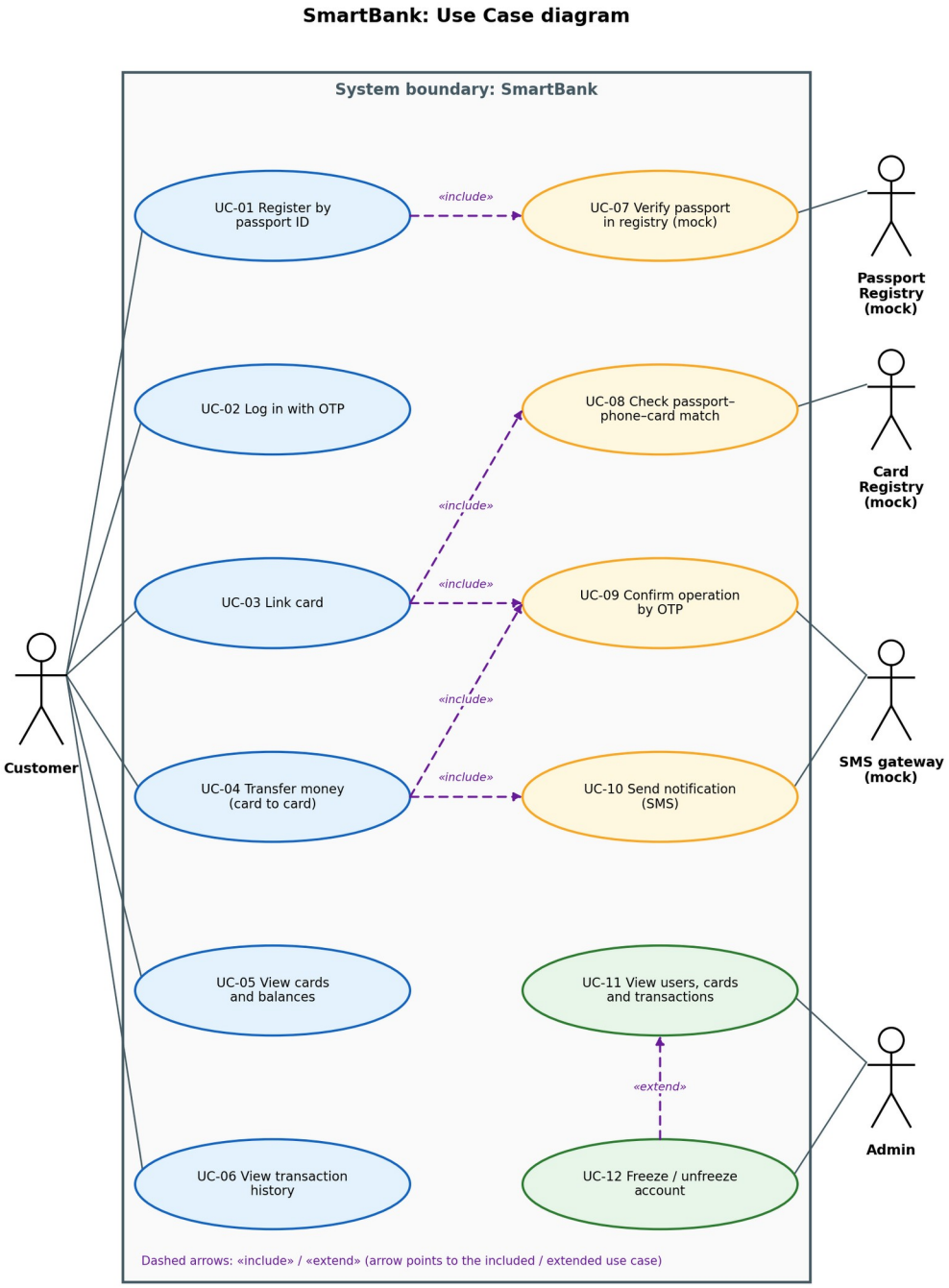


Figure 4 – Use Case diagram of SmartBank

Two main use cases are described below.

Table 17 – Use case UC-03 “Link card”

Field	Description
Name	UC-03 Link card
Actor	Customer (primary); Card Registry (mock) and SMS gateway (mock) as secondary actors
Precondition	The customer is registered (the phone number P came from the Passport Registry) and logged in.
Main flow	1. The customer opens “Add card” and enters the card number. 2. The system looks up the card in the Card Registry.

Field	Description
	3. The system checks (UC-08) that the card exists, is active, is not linked to another customer and that the phone attached to the card equals P. 4. The system sends an OTP to P through the SMS gateway (UC-09). 5. The customer enters the OTP. 6. The system links the card to the customer and shows it with a masked number and the balance.
Alternative flows	A1 (step 2): card not found or blocked – the system shows “card not available”. A2 (step 3): the phone attached to the card is different from P – the system rejects the linking and shows “the card is not attached to your phone number”. A3 (step 3): the card is already linked to another customer – the system rejects the linking. A4 (step 5): wrong OTP 3 times – the linking is cancelled; the customer can start again after 15 minutes. A5 (step 4): the OTP expired (3 minutes) – the customer requests a new code.
Outcome	The card is linked to the customer’s profile and can be used as the source of transfers.
Related requirements	FR-04, FR-05; NFR-03, NFR-08 (user stories US-04, US-05)

Table 18 – Use case UC-04 “Transfer money”

Field	Description
Name	UC-04 Transfer money (card to card)
Actor	Customer (primary); SMS gateway (mock, secondary)
Precondition	The customer is logged in, the account is not frozen and at least one linked card has funds. (In Sprint 1 the customer and the cards are seeded test data and the OTP step is added in Sprint 2.)
Main flow	1. The customer opens the transfer screen and selects the source card. 2. The customer enters the recipient card number and the amount. 3. The system checks that both cards are active, the amount is positive and the balance is sufficient. 4. The system shows a confirmation screen with the masked recipient card and the amount. 5. The customer confirms; the system sends an OTP to the customer’s phone (UC-09). 6. The customer enters the OTP. 7. The system writes the debit and credit in one database transaction, using an idempotency key. 8. The system shows a receipt with a reference number and sends SMS notifications to the sender and the recipient (UC-10).
Alternative flows	A1 (step 3): insufficient funds – the system returns HTTP 422 and shows “insufficient funds”; nothing is changed. A2 (step 3): recipient card not found or blocked – the system returns HTTP 404 and shows the reason. A3 (step 3): the account is frozen by an admin – the transfer is declined with the reason. A4 (step 6): wrong OTP 3 times – the transfer is cancelled; nothing is changed. A5 (step 7): the same idempotency key is received again (double click, retry) – the system returns the original receipt; no second debit. A6 (step 8): the SMS gateway is unavailable – the transfer stays valid and the notification is retried 3 times.
Outcome	The balance of the source card is decreased and the balance of the recipient card is increased by the same amount; a transaction record exists; both participants are notified.
Related requirements	FR-06, FR-07, FR-08, FR-10; NFR-03, NFR-06, NFR-08 (user stories US-01, US-07)

5.3. Technical Specification (TS)

The TS below follows the structure of GOST 34.602 and refers to the identifiers of Section 5.1. It is a short working version for the MVP; it is updated at the end of each Sprint.

1. General information

Table 19 – TS section 1 – general information

Item	Value
System name	SmartBank – digital banking system (code SB-2026)
Client	SmartBank JSC (hypothetical), pilot with 2,000 selected customers
Developer	Diyorxo'ja Saidaliyev, group 319-24 (individual project)
Deadlines	Start: week 1. Sprint 1 result: end of week 4. Complete MVP: end of week 10. System, security and acceptance testing: weeks 11–12. Pilot launch and handover: weeks 13–14.
Basis for development	Project charter (Table 4), Product Backlog (Table 12), requirements FR-01…FR-13 and NFR-01…NFR-08.

2. Purpose of the system and objectives of its development

Purpose: to let a person become a bank customer and make safe card-to-card transfers online, using only a passport ID, a phone number and OTP codes, and to give the bank an admin tool for monitoring. **Objectives:** (1) registration in ≤ 2 minutes; (2) at least 70 % of the pilot customers' transfers made digitally; (3) zero balance discrepancies; (4) transfer time ≤ 3 s; (5) full audit of admin actions. Success is measured by the criteria of the charter (Table 4).

3. Description of the object of automation

Environment. A commercial bank serves customers through branches. A customer has a passport, a phone number and one or more bank cards; each card is attached to a phone number. In the real world the passport data are stored in the national citizen registry and the card data in the bank's card-processing system. In this project these systems are **imitated by mock services** with seeded test data (Table 20). Admins use desktop computers; customers use smartphones.

Existing problems: (a) onboarding and simple transfers require a branch visit; (b) there is no instant check of the person by passport and phone; (c) customers are not told about operations immediately; (d) the bank has no simple tool to monitor users and transactions. Existing digital services (see Table 7) are widely used, but a bank that does not control its own reliable channel depends on third parties.

Table 20 – Mock external services and test data

Mock service	What it imitates	Data and behaviour
Passport Registry	National citizen / passport database; in reality also read with a camera (out of scope).	Table passports: passport_id (2 letters + 7 digits), full_name, birth_date, phone. 30 fictional citizens. API: GET /registry/passports/{id} returns the record or 404.
Card Registry	Card-processing system of the bank.	Table cards: card_number (16 digits), attached_phone, holder_name, balance (UZS), status (active / blocked), customer_id (empty until the card is linked). 40 cards; some have a phone that differs from the owner's passport phone, some are blocked, and 10 are already linked to 5 seeded test customers, to test all rejection cases. API: GET /cards/{number}.
SMS gateway	Telecom operator's SMS service.	POST /sms stores the message (phone, text, time) in a table and shows it in a developer inbox page; delivery can be switched off to test failures. OTP codes are read from this inbox during demos.

4. System requirements

4.1 Functional requirements: FR-01…FR-13 of Table 15. **4.2 Non-functional requirements:** NFR-01…NFR-08 of Table 16. Further requirements:

Table 21 – TS section 4 – hardware, software, interface and security requirements

Group	Requirement	Linked to
Hardware (server)	Pilot: one application and database server with 4 vCPU, 8 GB RAM, 100 GB SSD, located in Uzbekistan; staging server with 2 vCPU, 4 GB RAM. Daily backups to a second location.	NFR-02, NFR-05
Hardware (client)	Customers: any smartphone with 2 GB RAM and a modern browser. Admins: PC or laptop, screen ≥ 1280 px.	NFR-07
Software	Ubuntu LTS, Docker; Python 3.12 + FastAPI; PostgreSQL 16; Redis; React + TypeScript; Nginx; GitHub Actions.	NFR-05
Interface	Responsive web interface from 360 px; touch targets ≥ 44 px; font ≥ 16 px; text contrast ≥ 4.5:1; a transfer takes ≤ 5 steps; amounts always shown with currency; card numbers masked; every irreversible action has a confirmation screen. (Uzbek / Russian / English interface is a later release.)	NFR-04, NFR-07
Security	Roles: customer, admin (admins are created by the seed script); access by role only; OTP stored as a hash with expiry and attempt limit; rate limiting of OTP requests; HTTPS (TLS ≥ 1.2); encryption of passport ID and card number at rest; masking in the interface and logs; audit log of admin actions and personal-data views; consent of the customer to personal-data processing recorded at registration; data stored on servers in Uzbekistan; a threat model (STRIDE) and an OWASP ASVS checklist. A real launch would also require integration with the real registries and legal review (Central Bank rules, AML/KYC, Law No. ZRU-547).	FR-03, FR-07, FR-11, NFR-03

Group	Requirement	Linked to
Integration	The Passport Registry, Card Registry and SMS gateway are accessed through replaceable adapters (interfaces); mocks are used in this project and real services can be connected later without changing the business logic. The API is documented in OpenAPI.	FR-01, FR-04, FR-10

5. Scope and stages of work

Table 22 – TS section 5 – stages of work aligned with Sprints

Stage / Sprint	Weeks	Content (User Stories, tasks)	Result
Initiation	1–2	Charter, analysis, backlog, requirements, TS, kick-off page (this assignment)	Approved documents
Sprint 1	3–4	US-01 core (transfer without OTP) and the minimal card list of US-05; mock Card Registry; tasks T-02, T-04, T-05	Working card-to-card transfer with atomic update and idempotency
Sprint 2	5–6	US-01 OTP step (T-06), US-02 registration, US-03 login, US-04 card linking; mock Passport Registry and SMS gateway; tasks T-01, T-03, T-07, T-08	Full flow: register → login → link card → transfer
Sprint 3	7–8	US-05 complete, US-06 history, US-07 notifications, US-08 admin monitoring	Customer features and admin view complete
Sprint 4	9–10	US-09 freeze / unfreeze, US-10 unlink card; defect fixing, security hardening and performance tuning	Complete MVP
Testing and acceptance	11–12	System, load, security and concurrency testing; UAT with test customers and an admin	Test report, acceptance protocol
Pilot and handover	13–14	Deployment for the pilot group, admin guide, KPI measurement	System in operation, acceptance act

Estimates are initial and will be recalibrated after Sprint 1, when the real velocity is known.

6. Testing and acceptance procedure

- **Levels:** unit tests (pytest, Jest); API tests; UI tests (Playwright); manual test cases for every User Story (including all rejection cases of the passport–phone–card check); concurrency tests (50 and 1,000 parallel transfers); load test (Locust, 200 users); security scan (OWASP ZAP) and review against the threat model.
- **Every Sprint:** the Definition of Done (Section 4.5) is checked and a demo is given to the Product Owner and the client’s representative.
- **Acceptance:** the system is accepted when (a) all “Must” User Stories pass their acceptance criteria; (b) all NFR metrics are measured and met; (c) there are no open Critical or High defects and no High/Critical security findings; (d) UAT with at least 10 test customers and 1 admin gives SUS ≥ 75.

- **Procedure:** a test report is prepared; the client signs the acceptance act; open Medium and Low defects are written into the backlog.

7. Documentation requirements

The deliverables are: this TS; the Product Backlog and Trello archive; UML diagrams (use case, ER, architecture); OpenAPI description of the REST API and of the mock services; README with installation, seed data and deployment instructions; user guide for customers; admin guide; threat model and security checklist; test plan and test report; list of risks and decisions.

8. Development sources

GOST 34.602-2020 (technical specification for automated systems); ISO/IEC/IEEE 29148:2018 (requirements engineering); ISO/IEC 25010 (product quality model); the Scrum Guide 2020; OWASP ASVS (security requirements); Law of the Republic of Uzbekistan No. ZRU-547 “On Personal Data”; the project charter and the results of interviews with customers and bank staff; the competitor analysis of Section 2.3. The full list is in the References.

Task 6. Kick-off page

The page “Kick-off: SmartBank” is created in Google Docs (link in the list of working links) and has the content below. It is the single source of truth: a new team member or the client can understand in 5 minutes what we do and why.

Goal

The first version (MVP, pilot with 2,000 customers) must let a person register with a passport ID, link a card that matches their passport and phone, and make OTP-confirmed card-to-card transfers; it must give admins a monitoring panel and prove the effect with data: registration ≤ 2 minutes, at least 70 % of transfers made digitally, zero balance discrepancies, transfer time ≤ 3 s. **Sprint 1 goal:** a seeded test customer can see their cards and transfer money atomically and without duplicates.

Stakeholders and their expectations

Table 23 – Stakeholders and expectations

Stakeholder	Expectation from the first version
Client (SmartBank JSC board)	A visible shift of simple operations from branches to digital, a demonstration for investors, controlled cost and risk.
Customers	Instant registration with a passport ID, safe transfers confirmed by OTP, clear information about every operation.
Admins (bank staff)	A simple view of all users, cards and transactions, the ability to freeze an account, a full audit trail.
Marketer	A real working demo and screenshots to show; no promotion of features that do not exist yet.
Developer (the student)	A clear specification, stable scope inside a Sprint, realistic deadlines, clear rules for mock data.
Regulator (indirect)	Evidence of identity checks, audit trail and data protection if the system goes to real operation.

Scope and Out of Scope

Table 24 – Scope of the first version

In scope (MVP)	Out of scope (explicitly excluded)
Registration by passport ID (mock Passport Registry) with OTP Login with phone number and OTP Linking a card with the passport–phone–card check Viewing linked cards and balances Card-to-card transfer confirmed by OTP Transaction history SMS notification for each transaction Admin monitoring and account freezing Mock Passport Registry, Card Registry and SMS gateway	Passport scanning with a camera, biometrics Connection to real government and card-processing systems Real money and real SMS sending Bill payments, loans, international transfers Native iOS / Android applications Manual KYC review and fraud-analysis tools All “features of competitors” that are not in the backlog Paid advertising before the Sprint 1 demo

Risks and assumptions

Table 25 – Risk register

Risk	Impact	Response plan
Scope creep: the client or the marketer asks for “everything at once”	High – deadlines missed, poor quality	MoSCoW priorities; changes only at Sprint Planning; Out-of-scope list confirmed by the client.
Mock registries are different from the real passport and card systems	High – the integration later needs rework	Access only through adapters; data format agreed with the client; adapter tests; document differences.
Money errors: double debit, lost transfer or negative balance	High – loss of money and trust	Atomic transactions, idempotency keys, CHECK constraints; test T-08; NFR-06 measured before every release.
OTP and account security weaknesses (guessing, repeated requests, theft of a phone)	High – unauthorised transfers	Short OTP lifetime, attempt limits, rate limiting, hashing, masking, account freeze by an admin, OWASP ZAP scans.
Legal and regulatory duties not met in a real launch (licence, data law, AML/KYC)	High – a real launch is blocked	Prototype with test data only; consult a lawyer and the bank’s compliance officer before any real use.
Personal capacity (one person: exams, illness)	Medium – Sprint goal not reached	30 % buffer in every Sprint; strict MoSCoW – “Could” items are dropped first; weekly check of the plan.

Assumptions: (1) every person who registers has a passport record with a current phone number in the registry; (2) every card is attached to a phone number in the card registry; (3) customers have a smartphone and internet access and can receive SMS; (4) the baselines (1–2 days for onboarding, 45 % of transfers at branches) will be measured in the first pilot week; (5) the pilot uses test data and a mock SMS gateway until real services are connected.

Communication plan

Table 26 – Communication plan

Event / report	When	Participants and channel	Content
Daily stand-up	Mon–Fri, 5-minute written note	Student; note in Trello / Notion log	Yesterday, today, blockers
Sprint Planning	First day of a Sprint, 2 h	Student (Product Owner and Developer hats); written plan	Sprint goal, tasks, capacity, risk review
Weekly meeting with the client	Friday, 30 min	Student and the client's role-player; call or visit	Progress, decisions, risks, questions
Sprint Review (demo)	Last day of a Sprint, 1 h	Student, client's role-player, one test user; call or visit	Demo of working software, backlog re-prioritisation
Retrospective	After the Review, 15 min	Student; written note	What to start, stop and continue
Progress report to the client	Every Friday	Student → client; e-mail	One page: done, planned, risks, burndown screenshot, link to the demo
Documents and tasks	Continuous	Student	GitHub (documents, code), Trello (tasks), Google Docs (kick-off page)

Success metrics

2,000 registered pilot customers in 3 months; registration ≤ 2 minutes; at least 70 % of their transfers made digitally; 0 balance discrepancies; transfer time ≤ 3 s (p95); availability ≥ 99.5 %; SUS ≥ 75; Sprint goal reached in each Sprint.

Contribution of team members

This is an individual project; all work was done by the author.

Table 27 – Contribution of team members

Member	Work done	Contribution
Diyorxo'ja Saidaliyev	All work: charter, product and market analysis, process model, Scrum plan, backlog, requirements, use cases, technical specification, kick-off page, diagrams, data model and mock data design.	100 %

References

1. Schwaber K., Sutherland J. The Scrum Guide. 2020. https://scrumguides.org

2. Beck K. et al. Manifesto for Agile Software Development. 2001. https://agilemanifesto.org

3. Sommerville I. Software Engineering. 10th ed. Pearson, 2016.

4. Pressman R., Maxim B. Software Engineering: A Practitioner’s Approach. 9th ed. McGraw-Hill, 2019.

5. Cohn M. User Stories Applied. Addison-Wesley, 2004.

6. Kleppmann M. Designing Data-Intensive Applications. O’Reilly, 2017 (transactions, isolation, idempotency).

7. GOST 34.602-2020. Information technology. Set of standards for automated systems. Technical specification for the creation of an automated system.

8. ISO/IEC/IEEE 29148:2018. Systems and software engineering – Life cycle processes – Requirements engineering.

9. ISO/IEC 25010:2011. Systems and software engineering – Software product quality models.

10. OWASP Application Security Verification Standard (ASVS). https://owasp.org

11. Law of the Republic of Uzbekistan No. ZRU-547 “On Personal Data” of 2 July 2019 (with later amendments).

12. Click – electronic payment service (public description, Digital Government of Uzbekistan). https://dgov.uz/en/solution-detail/122

13. sayt.uz. Adding UzCard and Humo payments to your site via Payme, Click, Uzum. 20.11.2025. https://sayt.uz/en/blog/uzcard-humo-tolov-integratsiya

14. Stitch. Fintech in Uzbekistan (overview of Uzcard, Humo, Click, Payme). https://flow.stitch.money/post/fintech-in-uzbekistan

Note on tools: AI tools were used as a drafting assistant; all content was reviewed, checked and adapted by the author.